Using Cascading Style Sheets

Dr. Ahmad Mustafa

Assistant Professor

Department of Software Engineering

University of Sargodha

STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

What are Cascading Style Sheets?

- Cascading Style Sheets (CSS) were established by the World Wide Web Consortium (W3C). The CSS specification allows for more control over the look, or *style*, of web pages or other XML files by providing a central location, or *sheet*, where you can define how certain HTML (Hyper-Text Markup Language) or XML (eXtensible Markup Language) tags are going to be interpreted by the browser.

- Why is it called "cascading"? In CSS, multiple styles can be applied to a particular document (usually a web page or XML file). The browser will interpret these styles in a cascading fashion:
  - Style rules set up site-wide are overridden by styles located within individual pages.
  - Individual pages are overridden by styles inside an individual tag.
  - In addition, the end user can set up styles in the browser that will override the author's styles.
  - All matching rules for a particular selector will be applied, except where they conflict with each other (in which case, the latter rule would be applied, as determined by the cascade). In the following example, `<h2>` tags would be displayed in red and italics (but not blue):

```
h2 {font-style: italic;}
h2 {color: darkblue;}
h2 {color: red;}
```

What are Cascading Style Sheets? continued

- To properly see the effects of the Style Sheets, your visitors will need to use a web browser that is version 4.0 or newer. Fortunately, using CSS does not cause web pages to break when viewed on older browsers; however, the styles won't appear as defined. Since most people these days are using *Internet Explorer 6*, *Netscape 7*, or *Firefox 1* or newer, most browsers can properly display CSS.
- CSS-aware browsers apply their own stylesheet for every HTML element as the first set of rules in the cascade, and this set of rules form the default display for every element. For example, most browsers treat the `<p>` tag as a block element, as though there were the explicit declaration `p { display: block;}` By using CSS, you modify the default settings by overriding these implicit styles with an explicit declaration (for more on the block display, see slide 17).
- By using CSS, you can also control text formatting and location on the page. Using CSS can eliminate the need for tables as a layout tool. With CSS, logos can be created using just text, instead of having to rely on graphics. These changes make pages more accessible to a wider audience.
- CSS Specifications:
  - CSS 1: `http://www.w3.org/TR/REC-CSS1-961217.html`
  - CSS 2: `http://www.w3.org/TR/CSS2/`
  - CSS 2.1: `http://www.w3.org/TR/CSS21/`
- Differences between CSS 1, CSS 2, and CSS 2.1:
  - Between CSS 1 and CSS 2: `http://www.w3.org/TR/CSS2/changes.html`
  - Between CSS 2 and CSS 2.1: `http://www.w3.org/TR/CSS21/changes.html`

Pros and Cons of Using CSS

- Pros
  - Greater designer control of the appearance of the page
  - Easier management of site-wide changes
  - Greater accessibility to web sites by non-graphical browsers and web-page-reading software
- Cons
  - Different browsers may interpret Style Sheets in different ways
  - Some styles may not be seen at all on some browsers

CSS Examples

- The *CSS Zen Garden* shows some of the most advanced uses of CSS:

  `http://www.csszengarden.com/`

- *CSS in the real world: ajc.com's 'News Break'*

  `http://www.holovaty.com/blog/archive/2002/09/28/2340`

- *Web Standards* Tech Briefing – with CSS:

  `http://techbriefings.stanford.edu/web_standards/example1.html`

- *Web Standards* Tech Briefing – without CSS :

  `http://techbriefings.stanford.edu/web_standards/example2.html`

CSS Basics

- Under standard HTML, to create a web site with <h2> tags that have the standard features of a Header tag (that is, their own paragraph, bold, with a size change) and also are dark blue, you have to code each one as follows:

```
<h2><font color="darkblue">This is a darkblue H2 tag</font></h2>
```

- That's a lot of information to type every time you want to use a dark blue <h2> tag. Using CSS, all you need to do is type a regular <h2> tag. The style information will be included in the Style Sheet as follows:

```
h2 { color: darkblue;}
```

CSS Rules

- To change the color of ALL `<h2>` tags from darkblue to green, simply change the called for color to "green." The next time anyone sees the site, all the `<h2>` tags on all the pages will be green instead of darkblue.
- These styles are called *rules*. Each rule consists of a *selector* and a *declaration* (which is made up of a *property* and a *value)*.
- In the example below, *h2* is the selector, *color* is the property, and *darkblue* is the value. When used with web pages, selectors are usually HTML tags.

```
h2 { color: darkblue;}
```

- Syntax for a CSS rule:

```
selector { property: value; }
```

Grouping Styles and Selectors

- Each rule can include *multiple styles* by simply separating them by semicolons:

```
h2 { color: darkblue; font-style: italic;}
```

- Additionally, *multiple selectors* that have the same styles can be grouped by separating them with commas:

```
h1, h2, h3 { color: darkblue; font-style: italic;}
```

- *Contextual selectors* allow you to specify that something will change, but only when it is used in conjunction with something else. With the following style, `strong` will be displayed in red, but only when it occurs within `li` within `ul`.

```
ul li strong { color: red;}
```

  Elements being modified by contextual selectors need not appear immediately inside one another (using this style, **blah** would still be red text: `<ul><ol><li><strong> blah </strong></li></ol></ul>`).

- *Direct child selectors* allow you to specify that something will change, but only those that are immediately inside of another element. With the following style, only those `strong` elements that are directly inside of an `h1` will be purple; no `strong` tags deeper within the sheet will be purple.

```
h1 > strong { color: purple;}
```

- *Adjacent selectors* allow you to specify that something will change, but only when preceded by something else. With the following style, only those links (`a`) that are preceded by an `h2` will be green.

```
h2 + a { color: green;}
```

  Elements being modified by adjacent selectors appear immediately after one another. Using this style, this link would be green: `<h2>Visit Stanford!</h2><a href="http://www.stanford.edu">click here</a>`.
  This link would not: `<h2>Visit Stanford! <a href="http://www.stanford.edu">click here</a></h2>`.

- You can also group selectors *by attribute*. With the following style, centered h2 tags (<h2 align="center">) will be surrounded by a dotted border:

```
h2[align="center"] { border: dotted;}
```

Where do you put the styles?

- Style information can be located in three places:
  - External to the pages in a site
  - Internal to each page
  - Inline with individual tags
- Generally, creating an external style sheet file is the preferred method. To take full advantage of CSS, the Style Sheet for a site should be in this one external file, so that any changes will apply throughout the site. This also means that only one style document has to be downloaded for a single site.

Style Location: External

- The most common place to put style information is in an external document that each page of a web site points to directly.
- Any changes made to this single document will then be applied throughout the entire web site as each page is accessed by users.
- External Style Sheets have a .css extension.
- When linking to an external style sheet, you can also specify separate style sheets by media type:
  - **all** - Suitable for all devices.
  - **aural** - Intended for speech synthesizers.
  - **braille** - Intended for braille tactile feedback devices.
  - **embossed** - Intended for paged braille printers.
  - **handheld** - Intended for handheld devices (typically small screen, monochrome, limited bandwidth).
  - **print** - Intended for paged, opaque material and for documents viewed on screen in print preview mode.
  - **projection** - Intended for projected presentations
  - **screen** - Intended primarily for color computer screens.
  - **tty** - Intended for media using a fixed-pitch character grid, such as teletypes, terminals, or portable devices with limited display capabilities.
  - **tv** - Intended for television-type devices

External example

Text that appears in the *basic.css* style sheet document:

```
h2 {font-family: Arial, sans-serif; font-style: italic; color: green;}
p {font-family: Courier, monotype; font-style: bold; color: red; }
```

Text that appears in the *print.css* style sheet document:

```
h2 {font-family: Book Antiqua, Times, serif; font-style: italic; }
p {font-family: Courier, monotype; font-style: bold; }
```

HTML document, using the <link> tag method

```
<head>

<link rel="stylesheet" type="text/css"
    href="basic.css" media="all" />

<link rel="stylesheet" type="text/css"
    href="print.css" media="print" />

</head>
```

HTML document, using the @import and @media method

```
<head>

<style type="text/css">

<!--

@import url("basic.css") all;

@media url("print.css") print;

-->

</style>

</head>
```

Style Location: Internal

- Style information can also be included in the <head> section of an individual web page. This tends to work best when a single page needs to have a slightly different look than the rest of the site.

```
<head>
<style type="text/css">
<!--
h1 { font-family: Arial;
     font-style:italic;
     color:green;
   }
-->
</style>
</head>

<body>
<h1>This is sooooooooo cool!</h1>
<p>Nothing cool here.</p>
<h1>This one's cool, too!</h1>
</body>
```

This is sooooooooo cool!

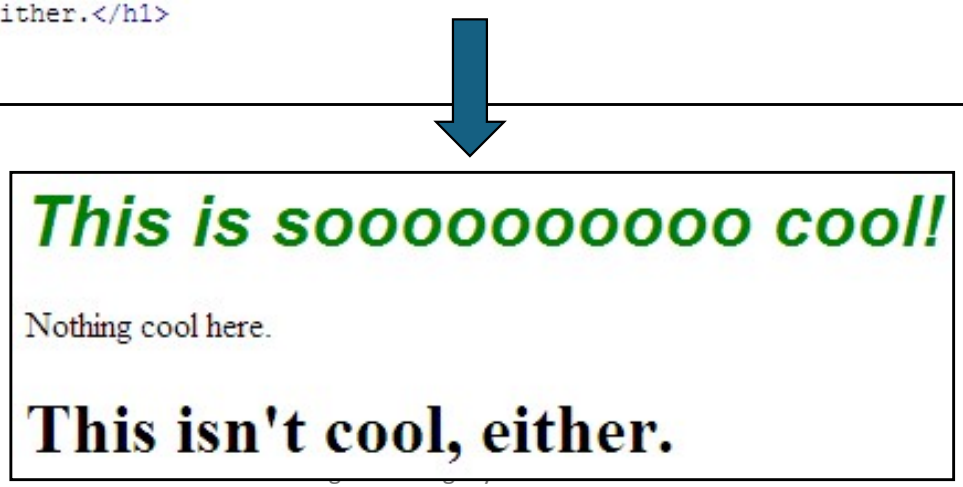
Nothing cool here.

This one's cool, too!

Style Location: Inline

- For extreme control, style information can be included in an individual tag. The style effects only that tag and no others in the document. This option is most useful for those rare occasions when a single tag needs to have a slightly different style.

```
<body>
<h1 style="font-family:Arial; font-style:italic; color:green;">This is sooooooooooo cool!</h1>
<p>Nothing cool here.</p>
<h1>This isn't cool, either.</h1>
</body>
</html>
```



Hierarchy of styles

- When style information is located in all three places in one site, the hierarchy is as follows:
  - External Style Sheets affect the entire site.
  - Internal styles affect only their own pages and override external styles.
  - Inline styles affect only their own tags and override both internal and external styles.

- For example, if an external Style Sheet sets `<h2>` tags to purple and a particular page has an internal style that changes that color to orange, the `<h2>` tags will be orange only on that page and nowhere else in the site. If there is a single `<h2>` tag on that page which specifies green as its color, then the color for that one tag will be green. All other `<h2>` tags on that page would be orange; the `<h2>` tags on the rest of the site would be purple.

!important

- Normally, the last rule listed in the cascade will take precedence over previous rules. In this example, the body font will be Verdana, not Times.

```
body  {font-family: Times;
       font-family: Verdana;}
```

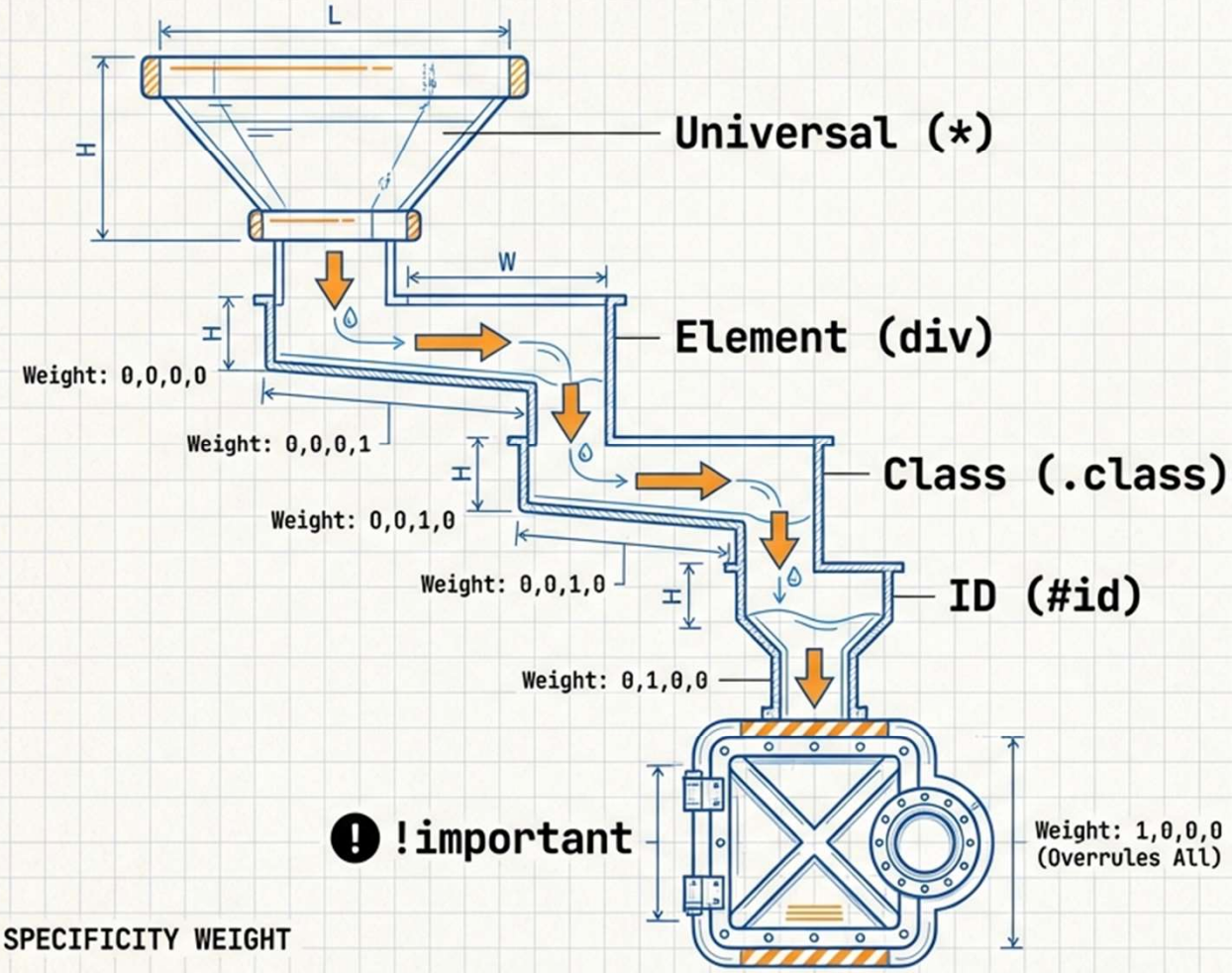
- However, by entering `!important` in a rule, that rule will take precedence, regardless of its location. In this example, the body font will be Times, not Verdana.

```
body  {font-family: Times !important;
       font-family: Verdana;}
```

Note: **!important** does not work with all properties in *Internet Explorer*.

STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

CSS CASCADE & SPECIFICITY: THE GRAVITY-FED SYSTEM



GRAVITY = SPECIFICITY WEIGHT

NOTE:

The Cascade prioritizes rules. Higher specificity wins.

!important acts as a gravity override, bypassing the normal specificity flow.

NotebookLM

Classes and IDs

- HTML has two attributes that make CSS even more useful: *class* and *ID*. They make it easy to apply style to just about any tag.
- Classes can describe a generic style that can be applied to any HTML element, or can be created for specific elements.
- When defining a style for elements with a particular class attribute in the Style Sheet, declare a rule using a dot (.) followed by the class name. To limit the style to a particular element with that class attribute, use a selector combining the tag name with a dot followed immediately by the class name.
  - The following rule would apply to any element with the attribute **class="shade"**

    ```
    .shade { background: yellow; }
    ```
  - The following rule would apply only to paragraph tags with the class **shade** (`<p class="shade">`)

    ```
    p.shade { background: red; }
    ```
- IDs are similar to classes, but IDs are unique – they can only be used with one instance of an element within a document.
- When defining a CSS rule using an ID-based selector, use a number/pound/hash sign (#) followed by the style name. To limit the style to a particular element with that id attribute, use a selector combining the tag name with a # and then the id name.
  - The following rule would apply to any element with the attribute **id="intro"**

    ```
    #intro { font-size: 2em; }
    ```
  - The following rule would apply only to heading 1 tags with the id **intro** (`<h1 id="intro">`)

    ```
    h1#intro { color: green; }
    ```

Class example

- Here's an example of a web page with an internal CSS style with a class called "highlight":

```
<head>
<style type="text/css">
<!--
.highlight {
    background-color: #CCCCCC;
}
-->
</style>
</head>

<body>
<p>This paragraph is not highlit.</p>
<p class="highlight">This paragraph is highlit.</p>
</body>
</html>
```

This paragraph is not highlit.

This paragraph is highlit.

STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

Inline vs. Block Display (HTML)

- All HTML elements (tags) are assigned a display property value of either inline or block.
- Inline elements display in browsers horizontally.

```
[INLINE ELEMENT 1]   [INLINE ELEMENT 2]   [INLINE ELEMENT 3]
```

- Block elements display in browsers vertically (stacked one on top of the other).

```
[BLOCK ELEMENT 1]
[BLOCK ELEMENT 2]
[BLOCK ELEMENT 3]
```

- Examples of inline elements:

```
<a>  <img>  <strong> <em>  <span>
```

- Examples of block elements:

```
<p>  <h1-h6>  <div>  <hr>  <table>  <ul>  <ol>
```

Inline vs. Block Display (CSS)

- Using CSS, you can change this inherent display property:
  - To force a block display, use the declaration `display: block;`
  - To force an inline display, use the declaration `display: inline;`
  - To force a list, use the declaration `display: list-item;`
  - To hide elements matching the selector, use the declaration `display: none;`

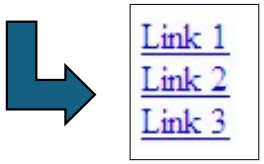
Example – display: block;

- Normally, <a> tags display inline.

```
<body>
<a href="link1.html">Link 1</a> <a href="link2.html">Link 2</a> <a href="link3.html">Link 3</a>
</body>
```

- But, if you add the style `a {display: block;}`, they will display as a vertical navigation menu:

```
<style type="text/cas">
<!--
a { display: block;}
-->
</style>
</head>
<body>
<a href="link1.html">Link 1</a> <a href="link2.html">Link 2</a> <a href="link3.html">Link 3</a>
</body>
```



Example – display: inline;

- Normally, the heading tags display in block format:

```
<body>
<h1>Heading 1</h1><h2>Heading 2</h2><h3>Heading 3</h3>
</body>
```

Heading 1

Heading 2

Heading 3

- But, to have them display inline, add the style `h1,h2,h3 {display: inline;}`:

```
<style type="text/css">
<!--
h1,h2,h3 { display: inline;}
-->
</style>
</head>
<body>
<h1>Heading 1</h1><h2>Heading 2</h2><h3>Heading 3</h3>
</body>
```

Heading 1Heading 2Heading 3

STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

Span and Div

- There are two tags that are particularly useful when using CSS: `<span>` and `<div>`. They are both container tags that have minimal formatting associated with them.
- The `<span>` tag is an inline element that simply holds text without doing anything special to it.
- The `<div>` tag is a block element and causes the text it encloses to start on a new line.
- Using `<span>` and `<div>` tags in conjunction with classes and IDs allows for great flexibility in creating pages.

Example using SPAN, DIV, Class, and ID

- Here's an example of a web page using a class, an id, and the span and div tags:

```
<head>
<style type="text/css">
<!--
#BrightText {color: #00ff00; font-weight: bolder;}
.PeriwinkleBlueText {color: #6699ff;}
-->
</style>
</head>

<html>
<body>
Baa, baa black sheep <br />
Have you any wool?<br />

<div id="BrightText">
Yes sir!  Yes sir!
</div>

Three bags full.  <br />
<span class="PeriwinkleBlueText">One</span> for my master.  <br />
<span class="PeriwinkleBlueText">One</span> for my dame.  <br />
<span class="PeriwinkleBlueText">One</span> for the little boy,  <br />
Who lives down the lane.
</body>
</html>
```

Baa Baa Black Sheep
Have you any wool?
Yes sir! Yes sir!
Three bags full.
One for the master
One for the dame
One for the little boy
Who lives down the lane.

Unit Measurements

- In CSS, you can measure units either in absolute values or in relative values.
- **Absolute values** are fixed, specific values. Since they are exact measurements, they allow the designer complete control over the display of the web pages.

  `mm, cm, in, pt, pc, xx-small, x-small, small, medium, large, x-large, xx-large`

- **Relative values** have no fixed, specific values, and are calculated in comparison to something else (usually the size of the default font or line size). Because different computers have different video cards, screen sizes, and users have differing eyesight abilities, relative values tend to be a better choice. They give the designer less absolute control but it often creates a better experience for the visitor.

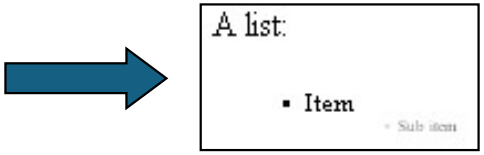
  `em, ex, px, larger, smaller, num%`

- Examples:

```
body { font-size: 12px; }
h1, h2, h3 { line-height: 200%;}
```

- Note – a warn ages: if you use percentages, and nest an element inside of that same element e cumulative.

```
<head>
<title>Untitled Document</title>
<style type="text/css">
ul {font-size: 70%}
</style>
</head>
<body>
A list:
<ul>
  <li>Item
    <ul>
      <li>Sub item</li>
    </ul>
  </li>
</ul>
</body>
```



Font and Text Styling

When choosing a font, there are several things to keep in mind:

1. Not everyone has the same set of fonts.
2. If you use a font that the visitor doesn't have, the page will display in the default font (usually Times), unless you provide more choices. To do this, add more than one font in your declaration, and always end with the font family (serif, sans-serif, or monospace):

   ```
   font-family: Verdana, Arial, Helvetica, sans-serif
   ```

3. Documents designed to be printed tend to look better in Serif fonts (Times, Georgia, Book Antiqua, etc.)
4. Documents designed to be viewed onscreen tend to look better in Sans-serif fonts (Verdana, Arial, Helvetica, etc.)

To apply a font to the entire web page, modify the body tag:

```
body {font-family: Verdana;}
```

To apply a font to a specific section of text, create a class, and use the span tag with that class:

```
.neatstuff {font-family: Comic Sans MS;}
<span class="neatstuff">This is in Comic Sans</span>
```

Modifying List Elements

- By default, unordered lists (`<ul>`) appear as bullets and ordered lists (`<ol>`) appear as numbers in HTML.
- Using CSS, you can modify how list items will appear:
  - Properties:
    `list-style`, `list-style-type`, `list-style-image`, `list-style-position`
  - Values:
    `disc`, `circle`, `square`, `decimal`, `decimal-leading-zero`, `lower-roman`, `upper-roman`, `lower-alpha`, `upper-alpha`, `lower-greek`, `lower-latin`, `upper-latin`, `hebrew`, `armenian`, `georgian`, `cjk-ideographic`, `hiragana`, `katakana`, `hiragana-iroha`, `katakana-iroha`, `none`, `url("graphic.gif")`
- Examples:

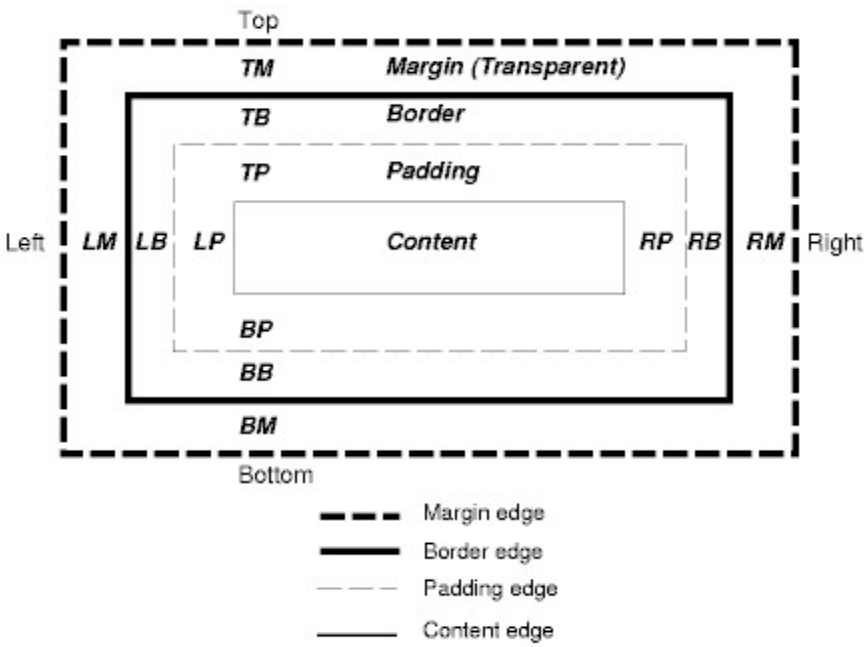
```
ul { list-style: disc; }
ol { list-style: upper-roman;}
li { list-style: url("blackball.gif");}
ul li { list-style-position: inside;}
```

The Box Model

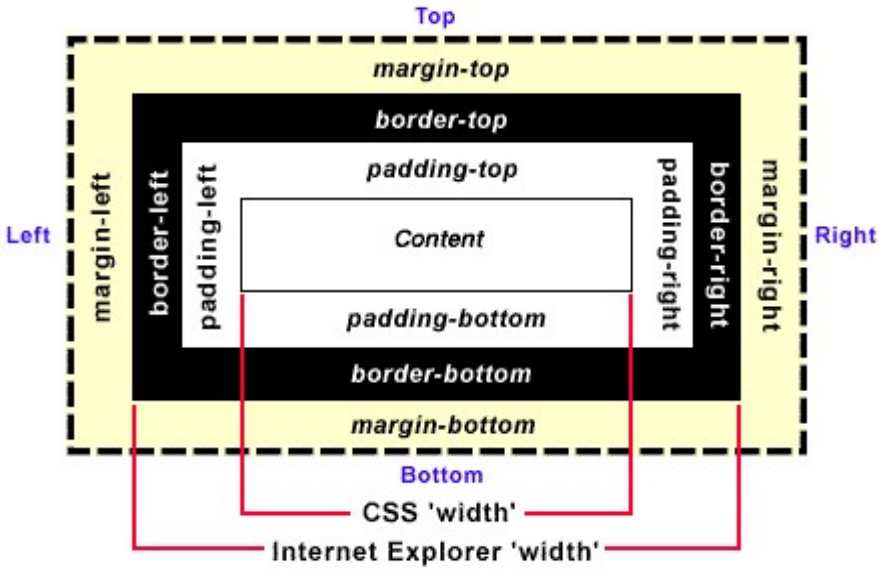
- When the browser draws an object on a page, it places it into an invisible rectangular space called a “bounding box.”
- You can specify the size, look, and feel of the **margins**, the **padding**, the **border**, and the content of the box.
- *Internet Explorer* interprets CSS box styles differently than most other web browsers.
- In CSS1, the **width** property is defined as the distance between the left and right edges of the bounding box that surrounds the element's content.
- Likewise, the **height** property is defined in CSS as the distance between the top and bottom edges of the bounding box.
- In *Internet Explorer*, however, the **width** and **height** properties also include the **border** and **padding** belts that surround the element's bounding box.

The Box Model: IE vs. CSS

CSS Standard



Internet Explorer



STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

Pseudo-elements and Pseudo-classes

- Two special predefined groupings, called pseudo-elements and pseudo-classes, are used in CSS to deal with special situations that do not exist with standard HTML. For example, under standard HTML, there is no way to automatically change the look and feel of the first letter or line of a paragraph. But by using the pseudo-element `:first-letter` you can specify a style that affects it:

```
p:first-letter { font-size: 200%; color:red;}
```

- Under standard HTML, there is no mechanism to deal with mouse movements. But with CSS, the pseudo-class `:hover` can be used to change the style of a link. In this example, `a:hover` is used to change the link color to red and the underlining to disappear whenever a mouse hovers over links:

```
a:hover {color: #ff0000; text-decoration: none;)
```

- To change the style of links, use the pseudo-class `:link`
  To change the style of visited links, use the pseudo-class `:visited`

```
a:link {color: #00f; font-weight: bold;)
a:visited {color: purple; border: groove;}
```

Positioning

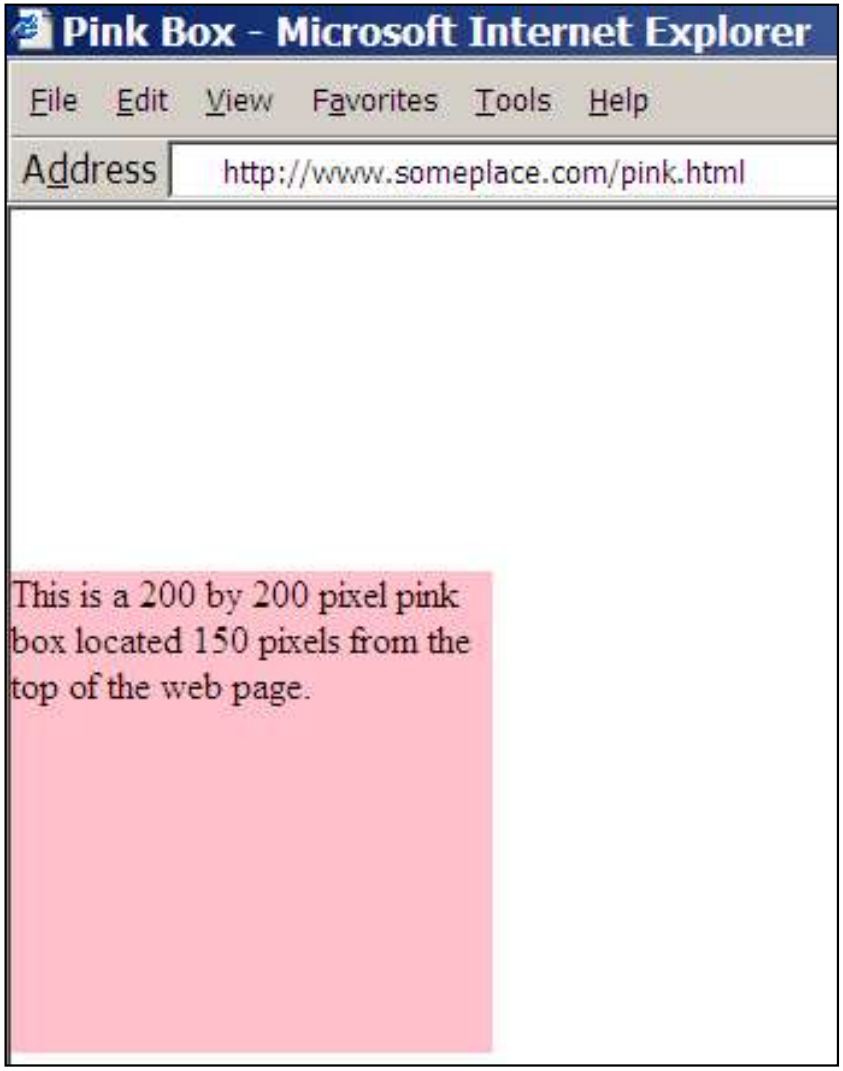
- Using CSS, you can place elements exactly on a page using a technique called "positioning." Positioning is determined by an X axis and Y axis. To specify a specific point on the screen, you can use the X and Y coordinate for that point.
- There are several ways to specify position in CSS: *absolute*, *relative*, *fixed*, *inherit*, and *static*.
- The three most often used are *absolute*, *relative*, and *fixed*.
- *Internet Explorer 6* only recognizes *absolute* and *relative*.

Absolute, Relative, Fixed, Inherit, and Static Positioning

- *Absolute positioning* defines the position of a given bounding box from the top and left side margins of the web page. This not only allows objects to be placed in an exact location, it also allows objects to be placed one on top of another.
- *Relative positioning* defines the positioning in such a way that elements are offset from the previous element in the HTML code. This allows objects to be placed in relation to one another.
- *Fixed positioning* defines the position of a given box relative to the window and remains in its specified location even as the content scrolls underneath it. This value does not work in Internet Explorer 6 or earlier. In IE 7, the browser must be in "standards-compliance mode".
- *Inherit positioning* explicitly sets the value to that of the parent (if the parent is *position:absolute*, the child will be *position:absolute*; if the parent is *position:fixed*, the child will be *position:fixed*).
- *Static positioning* is the default. It defines the position of a given box essentially as an unpositioned element – it flows in the normal rendering sequence of the web page.

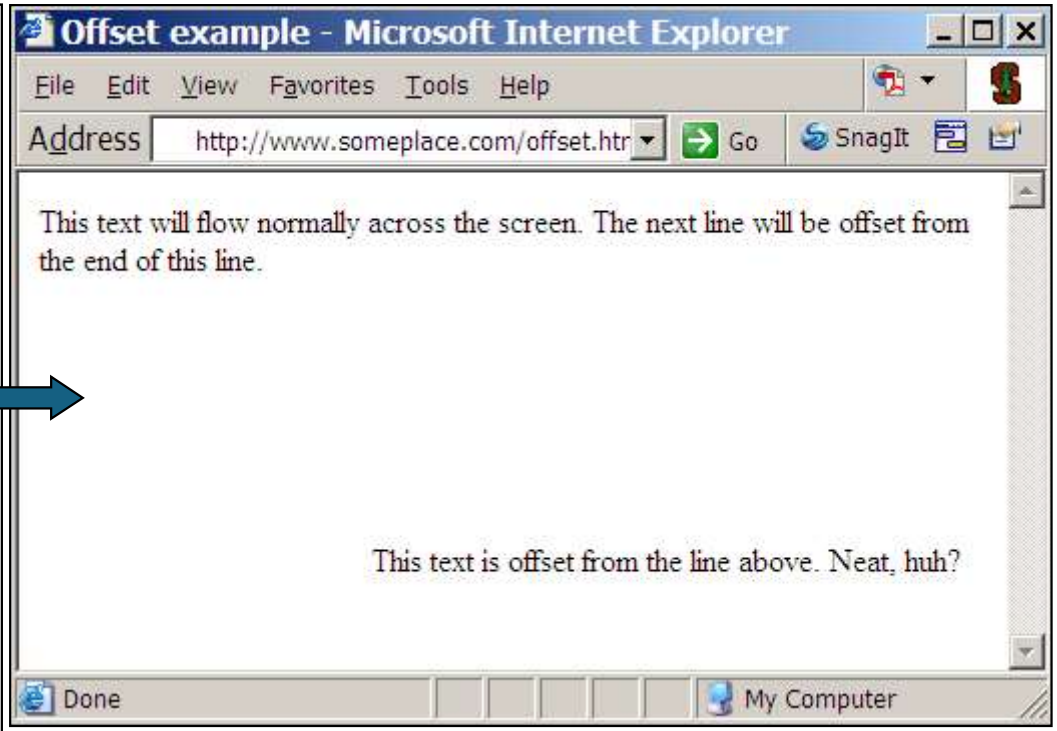
Absolute Positioning Example

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Pink Box</title>
<style type="text/css">
<!--
#pinkbox {
    position: absolute;
    width: 200px;
    background-color: pink;
    left: 0px;
    top: 150px;
    height: 200px;
    }
-->
</style>
</head>

<body>
<div id="pinkbox">This is a 200
by 200 pixel pink box located
150 pixels from the top of the
web page.
</div>
</body>
</html>
```



Relative Positioning Example

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Offset example</title>
<style type="text/css">
<!--
.offset {
    position: relative;
    top: 150px;
    left: 50px;
    }
-->
</style>
</head>

<body>
<p>This text will flow normally across
the screen.  The next line will be offset
from the end of this line.
<span class="offset">This text is offset
from the line above.  Neat, huh?</span>
</p>
</body>
</html>
```



Fixed Positioning – code view

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
     "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />

<title>Untitled Document</title>

<style type="text/css">

<!--

#links {

     position:fixed;

     border:dotted;

     border-color:#000000;

     width:20%;

     height:100%;

     z-index:1;

     left: 0px;

     top: 0px;

     background-color: #FFFFCC;

}

#main {

     position:absolute;

     left:25%;

     top:0px;

     width:70%;

     }

-->

</style>

</head>

<body>

<div id="main">

  <p>Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Quisque ultrices,   nibh ac
    rhoncus fermentum, orci sem dapibus nisi, sed tincidunt lectus lectus at   augue. In
    consectetuer vehicula enim. In hac habitasse platea dictumst. Donec a   nisl vitae tortor
    tristique viverra. Sed at lorem a ante lobortis molestie.   Nulla ullamcorper urna accumsan
    diam. Aliquam non eros. Pellentesque egestas   ultricies enim. Aenean lobortis, Nulla
    interdum commodo turpis. Sed ut mi id   elit vehicula sollicitudin. Sed lobortis, ligula sit
    amet euismod egestas, mi   ante iaculis nunc, ut rhoncus magna lectus ac arcu. In hac
    habitasse platea   dictumst. Proin quis ligula vitae quam pharetra adipiscing. Pellentesque
    tincidunt suscipit nibh. Ut fermentum suscipit justo. </p>

  <p>Fusce purus lectus, ultricies nec, aliquam at, facilisis id, arcu. Vestibulum   quis mi vel
    massa porta hendrerit. Nulla ullamcorper ligula nec lectus . Quisque   tempor, augue in
    molestie gravida, eros arcu luctus tortor, eu dignissim diam   urna sed urna. Ut dictum
    ultrices Iacus, in hac habitasse platea dictumst.   Suspendisse sed purus blandit metus
    ultricies suscipit. Proin diam justo,   consequat id, rhoncus eget, facilisis ut, lacus.
    Vivamus dignissim dui in justo.   Suspendisse elit. Nam nulla tortor, fringilla sed,
    faucibus quis, ullamcorper a,   leo. Fusce blandit condimentum turpis. Pellentesque vel odio
    et odio suscipit   egestas. Nullam ullamcorper sagittis ipsum. Maecenas fringilla malesuada
    pede.   Duis ut quam. </p>

</div>

<div id="links">

  <p>This area is fixed and will never move. It's good for things like navigation bars.</p>

  <ul>

    <li><a href="page1.html">Page 1</a></li>

    <li><a href="page2.html">Page 2</a></li>

    <li><a href="page3.html">Page 3</a></li>

    <li><a href="page4.html">Page 4</a></li>

    <li><a href="page5.html">Page 5</a></li>

  </ul>

</div></body></html>
```

Fixed Positioning – *Firefox* web browser

This area is fixed and will never move. It's good for things like navigation bars.

- Page 1
- Page 2
- Page 3
- Page 4
- Page 5

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Quisque ultrices, nibh ac rhoncus fermentum, orci sem dapibus nisi, sed tincidunt lectus lectus at augue. In consectetuer vehicula enim. In hac habitasse platea dictumst. Donec a nisl vitae tortor tristique viverra. Sed at lorem a ante lobortis molestie. Nulla ullamcorper urna accumsan diam. Aliquam non eros. Pellentesque egestas ultricies enim. Aenean lobortis. Nulla interdum commodo turpis. Sed ut mi id elit vehicula sollicitudin. Sed lobortis, ligula sit amet euismod egestas, mi ante iaculis nunc, ut rhoncus magna lectus ac arcu. In hac habitasse platea dictumst. Proin quis ligula vitae quam pharetra adipiscing. Pellentesque tincidunt suscipit nibh. Ut fermentum suscipit justo.

Fusce purus lectus, ultricies nec, aliquam at, fac[illegible]s id, arcu. Vestibulum quis mi

Done

This area is fixed and will never move. It's good for things like navigation bars.

- Page 1
- Page 2
- Page 3
- Page 4
- Page 5

urna. Ut dictum ultrices lacus. In hac habitass[illegible]a dictumst. Suspendisse sed purus blandit metus ultricies suscipit. Proin diam [illegible]sto, consequat id, rhoncus eget, facilisis ut, lacus. Vivamus dignissim dui in justo. Suspendisse elit. Nam nulla tortor, fringilla sed, faucibus quis, ullamcorper a, leo. Fusce blandit condimentum turpis. Pellentesque vel odio et odio suscipit egestas. Nullam ullamcorper sagittis ipsum. Maecenas fringilla malesuada pede. Duis ut quam.

Donec erat. Mauris enim nibh, ornare quis, ullamcorper nec, consectetuer ut, urna. Fusce id nulla in orci faucibus dictum. Praesent aliquet tempus purus. In mi ligula, facilisis ac, mollis sed, sodales at, magna. Nullam pharetra. Nullam ac lorem sit amet augue porta consequat. Donec justo erat, rhoncus congue, consequat a, malesuada ac, leo. Suspendisse potenti. Aliquam tempus elementum ipsum. Praesent eu felis. In adipiscing

Done

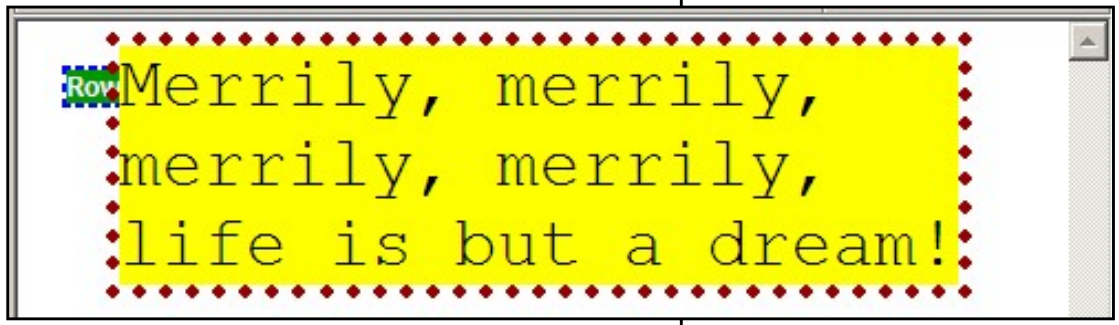
STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

Layers and the “Bounding Box”

- When the browser draws an object on a page, it places it into an invisible rectangular space called a “bounding box.” You can set the box’s exact location on the page or offset it from other objects on the page. As mentioned in the previous slides, you can also specify the size of the box.
- With CSS, these boxes can be stacked one on top of another as layers. Horizontal and vertical positioning happen along the X and Y axes, and the layered positioning happens along the Z axis.
- The Z axis is set using the CSS style `z-index`, which allows you to specify which layer appears on top of the others. By setting the `z-index` higher or lower, an object can move up and down a stack. The higher the `z-index`, the more “on top” it is.

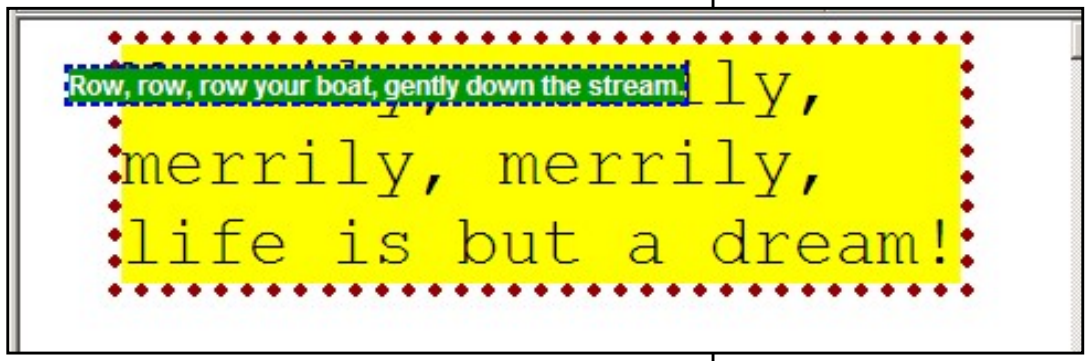
Layering Example 1

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Untitled Document</title>
<style type="text/css">
<!--
.over {
        font-family: "Courier New", Courier, mono;
        font-size: 200%;
        color: #000000;
        background-color: #FFFF00;
        border: thick dotted #990000;
        position: absolute;
        left: 40px;
        top: 5px;
        z-index: 2;
}
.under {
        font-family: Arial, Helvetica, sans-serif;
        font-size: 75%;
        font-weight: bold;
        color: #FFFFFF;
        background-color: #009900;
        border: thin dashed #0000FF;
        position: absolute;
        left: 20px;
        top: 20px;
        z-index: 1;
}
-->
</style>
</head>

<body>
<p class="under">Row, row, row your boat, gently down the stream.</p>
<p class="over">Merrily, merrily, merrily, merrily, life is but a dream!</p>
</body>
```



Layering Example 2

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Untitled Document</title>
<style type="text/css">
<!--
.over {
        font-family: "Courier New", Courier, mono;
        font-size: 200%;
        color: #000000;
        background-color: #FFFF00;
        border: thick dotted #990000;
        position: absolute;
        left: 40px;
        top: 5px;
        z-index: 1;
}
.under {
        font-family: Arial, Helvetica, sans-serif;
        font-size: 75%;
        font-weight: bold;
        color: #FFFFFF;
        background-color: #009900;
        border: thin dashed #0000FF;
        position: absolute;
        left: 20px;
        top: 20px;
        z-index: 2;
}
-->
</style>
</head>

<body>
<p class="under">Row, row, row your boat, gently down the stream.</p>
<p class="over">Merrily, merrily, merrily, merrily, life is but a dream!</p>
</body>
```



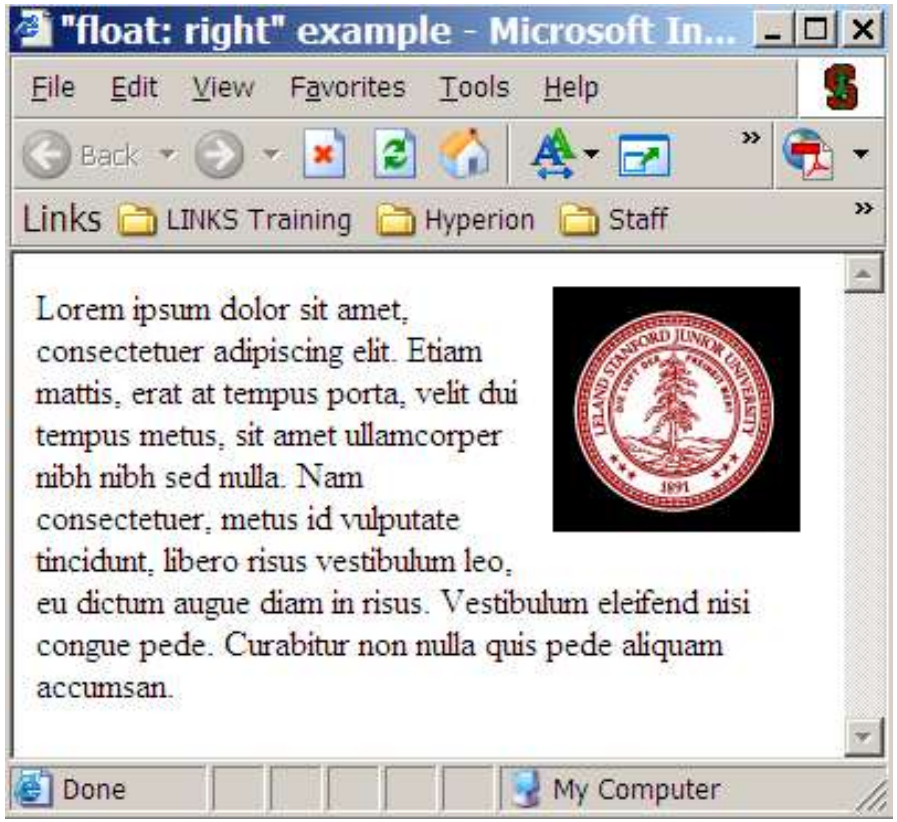
Float

- If you want to wrap content around other content (such as text around a picture), you can use the `float` property.
- The `float` property determines on which side of the bounding box the element aligns so that the other content wraps around it.

STANFORD UNIVERSITY • INFORMATION TECHNOLOGY SERVICES

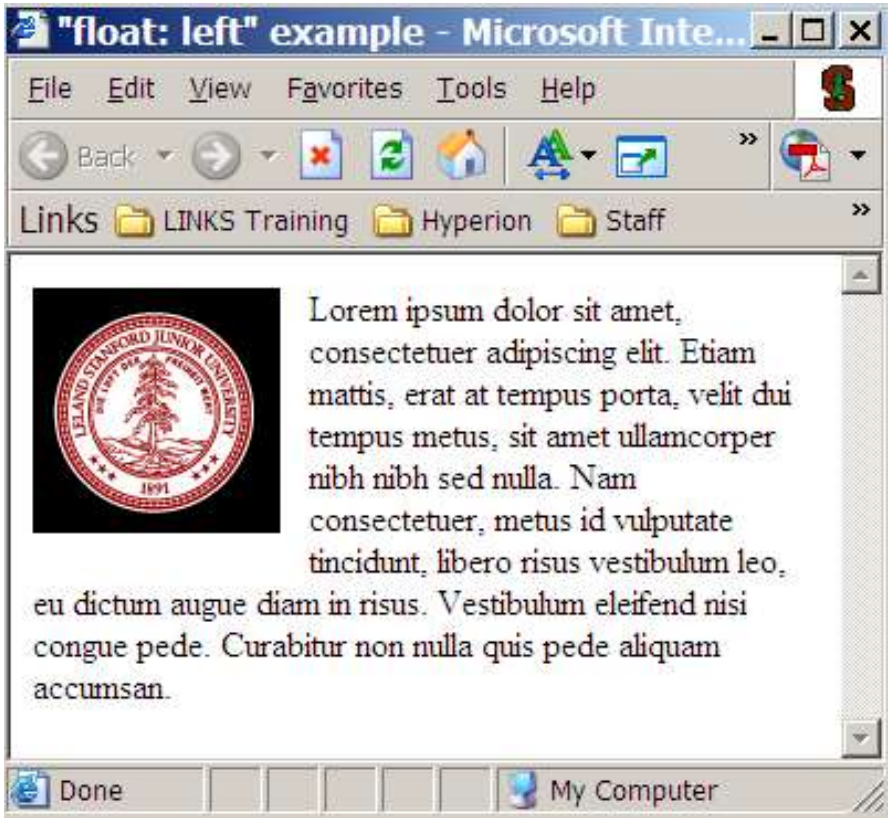
Float Example 1 – float: right

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en"
lang="en">
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" />
<title>&quot;float: right&quot; example</title>
<style type="text/css">
    #floatright { float: right; }
</style>
</head>
<body>
<div id="floatright"><img src="down_seal3.gif" width="122"
height="118" alt="Stanford Seal" /></div>
<p> Lorem ipsum dolor sit amet, consectetuer adipiscing
elit. Etiam mattis,  erat at tempus porta, velit dui tempus
 metus, sit amet ullamcorper nibh  nibh sed nulla. Nam
consectetuer, metus id vulputate tincidunt, libero  risus
vestibulum leo, eu dictum augue diam in risus. Vestibulum
eleifend nisi congue pede. Curabitur non nulla quis pede
aliquam  accumsan. </p>
</body>
</html>
```



Float Example 2 – float: left

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en"
lang="en">
<head>
<meta http-equiv="Content-Type" content="text/html;
charset=iso-8859-1" />
<title>&quot;float: left&quot; example</title>
<style type="text/css">
    #floatleft { float: left;   }
</style>
</head>
<body>
<div id="floatleft"><img src="down_seal3.gif" width="122"
height="118" alt="Stanford Seal" /></div>
<p> Lorem ipsum dolor sit amet, consectetuer adipiscing
elit. Etiam mattis,  erat at tempus porta, velit dui tempus
metus, sit amet ullamcorper nibh  nibh sed nulla. Nam
consectetuer, metus id vulputate tincidunt, libero  risus
vestibulum leo, eu dictum augue diam in risus. Vestibulum
eleifend nisi congue pede. Curabitur non nulla quis pede
aliquam  accumsan. </p>
</body>
</html>
```



Resources

- A List Apart – articles on practical issues and suggestions for working with CSS correctly
  `http://www.alistapart.com/topics/code/css`
- Example XHTML Pages, with and without the CSS Style Sheet:
  `http://techbriefings.stanford.edu/web_standards/example1.html`
  `http://techbriefings.stanford.edu/web_standards/example2.html`
  `http://techbriefings.stanford.edu/web_standards/example.css`
- The *CSS Zen Garden* shows some of the most advanced uses of CSS: `http://www.csszengarden.com/`
- *CSS in the real world: ajc.com's 'News Break':*
  `http://www.holovaty.com/blog/archive/2002/09/28/2340`
- Microsoft's CSS Information:
  `http://msdn.microsoft.com/workshop/author/css/reference/attributes.asp`
- Microsoft's Style Sheet Demonstrations:
  `http://www.microsoft.com/typography/css/gallery/extract1.htm`
  `http://www.microsoft.com/typography/css/gallery/slide1.htm`
- W3C Style Examples
  `http://www.w3.org/Style/Examples/007`
- W3C CSS 2.1 Specifications:
  `http://www.w3.org/TR/CSS21/`
- W3Schools CSS Tutorial:
  `http://www.w3schools.com/css`
- W3Schools CSS Reference:
  `http://www.w3schools.com/css/css_reference.asp`
- Webmonkey's Cascading Style Sheet Guide:
  `http://www.webmonkey.com/reference/stylesheet_guide/`
- Brian Wilson's Cascading Style Sheet Reference Guide:
  `http://www.blooberry.com/indexdot/css/index.html`